

# sv-tpu-core

## Specification Document — REV 2 (As-Built)

*Systolic Array Matrix Multiply Unit — 4×4 physical, DiP dataflow, weight-stationary, int8*

**Owner: Atharva Kulkarni (verification lead) | Revision date: July 29, 2026 | Supersedes: SpecDoc v1.0 (Phase 1, Jul 1–8 2026)**

### WHY THIS REVISION EXISTS

v1.0 of this document was written in Phase 1, before any RTL existed, and locked a design (horizontal-flow weight-stationary array, 2N pipelined latency) that the team has since departed from. The repository has moved to a DiP (Diagonal-input, Permuted weight-stationary) dataflow, and the measured latency contract is no longer 2N. This revision replaces v1.0 as the single source of truth and describes the design AS IT ACTUALLY EXISTS IN THE REPO TODAY, plus what remains open. Where v1.0's assumptions turned out to be wrong, that is called out explicitly rather than silently corrected — some of this is genuinely still unresolved and needs a team decision, not a spec-writer's guess.

## 0. How to Read This Document

This spec has four parts, same skeleton as v1.0:

- **Part A — Design Spec:** the dimensions, dataflow, and timing of the hardware as built.
- **Part B — Register (RAL) Spec:** the three control registers. Unchanged from v1.0 — this part was never in question.
- **Part C — Performance Spec:** how fast the design actually runs, and the open latency-contract question.
- **Part D — Formal Verification Spec:** the three JasperGold proofs, their current status, and how the DiP rewrite affects them.

### 0.1 Status Legend Used Throughout

| Marker      | Meaning                                                                                                     |
|-------------|-------------------------------------------------------------------------------------------------------------|
| DONE        | Implemented, exercised in regression, matches this document.                                                |
| IN PROGRESS | Actively being worked; scaffolding may exist but the deliverable isn't complete.                            |
| OPEN ITEM   | A real, unresolved decision the team needs to make — not a typo, not fixed by this document.                |
| REMOVED     | Existed in a prior revision or is still physically present in the repo but is no longer part of the design. |

### 0.2 What Changed Since v1.0 — Executive Summary

Two structural changes and one open timing question separate this revision from v1.0:

- **1. Dataflow architecture replaced:** the array moved from the originally-planned horizontal-activation-flow weight-stationary design to **DiP (Diagonal-input, Permuted weight-stationary)**, per Abdelmaksoud, Agwa & Prodromakis, arXiv:2412.09709v3. This is a real interconnect rewrite, not a renaming — see Part A.
- **2. skew\_buffer.sv is dead:** its job (staggering each row's activation feed) is now performed internally by DiP's diagonal interconnect. It is replaced by `deskew_capture.sv`, which captures the OUTPUT side instead. The file still exists in `rtl/` but is not instantiated anywhere — see A.4.
- **3. Latency is an OPEN ITEM, not 2N:** the RTL as built, the controller SVA, and the scoreboard all independently converge on **active\_dim + N + 1** cycles (equivalently `dim + 5` for `N=4`), not 2N. `mmu_perf_checker.sv` was NOT updated and still hardcodes 2N — this is a live discrepancy inside the current regression, not a documentation lag. See Part C.

### 0.3 Category & SVA Completion Status (per your note)

Captured here so this document doesn't silently claim completeness that doesn't exist yet:

| Item                                          | Status | Detail                                                              |
|-----------------------------------------------|--------|---------------------------------------------------------------------|
| UVM Category 1 — Basic Functional Correctness | DONE   | <code>mmu_cat1_tests.sv</code> — TC-001 through TC-010 implemented. |
| UVM Category 2 — Back-to-Back Sequencing      | DONE   | <code>mmu_cat2_tests.sv</code> — TC-011 through TC-013 implemented. |

| Item                                                 | Status                                  | Detail                                                                                                                                                                           |
|------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UVM Category 3 — Weight Poison & Reset Stress        | IN PROGRESS                             | weight_poison_seq exists as a stimulus sequence in mmu_sequences.sv. No mmu_cat3_tests.sv test-class file and no reset_stress_seq yet.                                           |
| UVM Category 4 — Reset Behavior                      | IN PROGRESS                             | No test-class file yet (TC-021, TC-022, TC-034 not implemented).                                                                                                                 |
| UVM Category 5 — Illegal Operation / Error Injection | DONE                                    | mmu_cat5_tests.sv — TC-023 through TC-026 implemented.                                                                                                                           |
| UVM Category 6 — Latency & Throughput Performance    | DONE (mechanically) / OPEN (contract)   | mmu_cat6_tests.sv implemented and runs, but see the latency OPEN ITEM in Part C before trusting its pass/fail against 2N.                                                        |
| RAL Built-In Sequences (TC-032, TC-033)              | DONE                                    | mmu_ral_and_negative_tests.sv.                                                                                                                                                   |
| TC-035 Force-Injection Series                        | DONE for a/b/c/d/e/f                    | mmu_ral_and_negative_tests.sv implements TC-035a, b, cd (c and d share a base class), e, f. TC-035g (D2 / throughput SVA) has no assertion to target yet — see next row.         |
| A1 — axi_awvalid_stable (axi_lite_sva.sv)            | DONE                                    | Implemented; note the property changed from v1.0's stated form — see A.9 note.                                                                                                   |
| A2 — axi_wvalid_with_awvalid (axi_lite_sva.sv)       | REVISED, DONE                           | No longer requires simultaneous awvalid/wvalid (that was never a real AXI-Lite requirement); now checks wdata/wvalid stability the same way A1 checks awvalid. See D-note below. |
| B1–B4 (mmu_controller_sva.sv)                        | DONE                                    | All four implemented; B3's property text was rewritten for the DiP-era latency formula — see Part C.                                                                             |
| C1 — zero_input_no_accumulate (pe_sva.sv)            | NOT STARTED                             | File does not exist in the repo yet. Bound module, positive/negative test mapping, and TC-035f's target are all still pending this file.                                         |
| D1 / D2 (mmu_perf_checker.sv)                        | DONE (D1, but stale) / NOT WRITTEN (D2) | D1 exists but asserts the stale 2N formula (see Part C OPEN ITEM). No back-to-back throughput property (D2) is implemented yet, so TC-035g has nothing to target.                |
| Formal Proof 1 — Overflow Impossibility              | DONE — PROVEN                           | formal/results/pe_overflow_report.txt.                                                                                                                                           |
| Formal Proof 2 — AXI Compliance                      | DONE — PROVEN                           | formal/results/axi_compliance_report.txt.                                                                                                                                        |
| Formal Proof 3 — 2x2 Functional Correctness          | DONE — PROVEN (bounded)                 | See Part D; mmu_formal.sv deliberately takes no position on the latency OPEN ITEM.                                                                                               |

## Part A — Design Spec (As Built)

This part defines what the chip actually is today. Where v1.0 locked a design that was never built this way, that is called out rather than silently patched over.

### A.1 What the Design Does (plain language)

The design multiplies two small grids of numbers together — a matrix multiply,  $C = A \times B$ . It is a systolic array: a grid of small multiply-add units (PEs) that pass data to neighboring PEs every clock cycle instead of going back and forth to memory.

Weights are loaded into the grid once and held stationary (unchanged from v1.0). What has changed is HOW activations move through the grid and how results are read back out — see A.2 and A.5.

### A.2 Architecture Change: DiP (Diagonal-Input, Permuted Weight-Stationary)

#### ARCHITECTURE CHANGE — SUPERSEDES v1.0

v1.0 specified a horizontal-flow design: activations enter from the left edge, one column per cycle, and flow rightward through each row; results drain out the bottom via a vertical accumulator chain. The repo has since moved to DiP (Diagonal-input, Permuted weight-stationary), per Abdelmaksoud, Agwa & Prodromakis, "DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration" (arXiv:2412.09709v3), Section III-A/III-B. This is implemented in rtl/pe.sv and rtl/systolic\_array.sv.

What DiP changes, concretely:

- **Row 0 is the only external entry point.** A FULL ROW of matrix A (all N elements) is fed in parallel into PE[0][0..N-1] every active cycle — not one element trickling in per row per cycle as in the old design.
- **Diagonal downward movement.** For row > 0: PE[row][col].activation\_in comes from PE[row-1][(col+1) mod N].activation\_out. Each PE's activation output is still registered (one cycle of latency per hop — unchanged principle from v1.0's pipeline register), but the neighbor it feeds is diagonal, not horizontal.
- **Weight permutation (DiP Algorithm 1).** permuted[row][col] = weights[(row+col) mod N][col], applied combinationally inside systolic\_array.sv. Upstream logic (mmu\_top.sv, the data agent) continues to hand the array the natural, unpermuted weight matrix — permutation is entirely internal.
- **Vertical accumulator chain is unchanged.** PE[row][col].accum\_in still comes from PE[row-1][col].accum\_out, row 0's accum\_in still ties to 0. Partial sums still drain out the bottom row exactly as in v1.0.

pe.sv itself needed zero structural changes for DiP — it still has exactly the three registers it always had (weight register, one activation pipeline register, one accumulator). The rewiring is entirely inside systolic\_array.sv's interconnect; pe.sv has no notion of who its neighbor is.

### A.3 Locked Design Decisions

| Decision            | Value                                                   | Status vs v1.0                           |
|---------------------|---------------------------------------------------------|------------------------------------------|
| Architecture        | Weight-stationary systolic array, DiP diagonal dataflow | CHANGED — v1.0 specified horizontal flow |
| Physical array size | 4x4 processing elements (16 PEs)                        | Unchanged                                |
| Parameterized on N  | Yes — one constant changes 4x4 to 8x8                   | Unchanged                                |

| Decision                             | Value                                     | Status vs v1.0                                        |
|--------------------------------------|-------------------------------------------|-------------------------------------------------------|
| Active size at runtime               | DIM_REG selects N (1–4)                   | Unchanged                                             |
| Activation data type                 | int8 (signed, –128...127)                 | Unchanged                                             |
| Weight data type                     | int8 (signed, –128...127)                 | Unchanged                                             |
| Accumulator data type                | int32 (signed)                            | Unchanged — see A.10 for the corrected overflow bound |
| Control interface                    | AXI-Lite slave, 3 registers, control only | Unchanged                                             |
| Systolic Data Setup (skew_buffer.sv) | REMOVED from instantiation                | CHANGED — see A.4                                     |

## A.4 Module List and Boundaries

The DUT boundary is still mmu\_top.sv. Two modules changed role since v1.0:

| Module            | Role                                                                                                                                                                                                                | Status                                                            |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| mmu_top.sv        | Top wrapper. Instantiates everything below. DUT boundary.                                                                                                                                                           | Updated instance list (skew_buffer removed, deskew_capture added) |
| axi_lite_slave.sv | Control panel — decodes the three AXI-Lite registers (Part B).                                                                                                                                                      | Unchanged                                                         |
| mmu_controller.sv | Sequencer FSM. ACTIVATION_FLOW sizing changed for the DiP capture window — see A.5/Part C.                                                                                                                          | Timing logic changed                                              |
| systolic_array.sv | N×N grid of PEs. Interconnect rewired for DiP diagonal dataflow (A.2). Applies weight permutation internally.                                                                                                       | Rewired for DiP                                                   |
| pe.sv             | One processing element. Structurally unchanged — same three registers as always.                                                                                                                                    | Unchanged internally                                              |
| deskew_capture.sv | NEW. Snapshots the array's raw, still-accumulating bottom row into a proper N×N result matrix at the correct per-output-row cycle. Replaces skew_buffer.sv's role (staggering) with an output-side capture instead. | NEW — replaces skew_buffer.sv                                     |
| skew_buffer.sv    | Formerly staggered activations before the array (v1.0's design). Under DiP the diagonal interconnect performs this staggering internally; feeding pre-skewed activations would double-stagger every row past row 0. | REMOVED — file still present in rtl/ but not instantiated         |
| output_buffer.sv  | Holds the int32 results until the testbench reads them after done. Timing bug fixed (see A.4 note below) so results are combinationaly valid the same cycle done asserts, not one cycle late.                       | Bug fix applied, otherwise unchanged                              |
| mmu_if.sv         | SystemVerilog interface bundle connecting the DUT to the testbench.                                                                                                                                                 | Unchanged                                                         |

*output\_buffer.sv note: an earlier version gated its capture register directly on done, which — because done is itself a registered signal — introduced a spurious extra cycle of latency on top of deskew\_capture's already-correct timing. This was fixed by splitting into a combinational passthrough (valid the same cycle done is high) plus a held register for subsequent*

*cycles. This fix is already in the repo; it is documented here because it is exactly the kind of off-by-one that the latency OPEN ITEM in Part C needs to be checked against.*

## A.5 Controller FSM and Capture Timing

The controller (mmu\_controller.sv) is unchanged in its five states, but ACTIVATION\_FLOW's duration and DONE's trigger condition were both resized for DiP's capture timing.

| State           | What happens                                                                                                                                  | Leaves when                                                    |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| IDLE            | Waiting. Nothing runs.                                                                                                                        | CTRL_REG start bit is written (=1) with a legal dimension.     |
| WEIGHT_LOAD     | Weights are loaded into the PEs (permuted internally by systolic_array.sv) and held stationary.                                               | After exactly N cycles (WEIGHT_LOAD_CYCLES = N).               |
| PE_CLEAR        | pe_clear is asserted for exactly one cycle, zeroing every PE accumulator.                                                                     | After exactly one cycle.                                       |
| ACTIVATION_FLOW | Full rows of matrix A flow into row 0 every cycle; deskew_capture.sv snapshots each output row as it settles into the bottom accumulator row. | When the internal counter reaches active_dim + N (see Part C). |
| DONE            | done bit (STATUS_REG) is set. Results are valid in the output buffer.                                                                         | start is de-asserted / next run begins.                        |

### OPEN ITEM — see Part C for the full derivation

The ACTIVATION\_FLOW duration is sized in the RTL as active\_dim + N cycles (flow\_last = dim\_q + N), which — combined with deskew\_capture's one-cycle capture lag — produces a measured start-to-done latency of active\_dim + N + 1 cycles, NOT the 2N that v1.0 locked and that mmu\_perf\_checker.sv still asserts. This is flagged in three independent places in the repo (mmu\_controller.sv's own header comment, mmu\_controller\_sva.sv's B3 property, and a removal note in mmu\_scoreboard.sv)

| State                                                                                  | What happens | Leaves when |
|----------------------------------------------------------------------------------------|--------------|-------------|
| and is a genuine open decision, not a spec-writer's assumption. Full detail in Part C. |              |             |

## A.6 pe\_clear — still the signal most likely to cause a bug

Unchanged from v1.0: pe\_clear must be asserted for EXACTLY ONE cycle, only in the PE\_CLEAR state, fanned out to all 16 PEs simultaneously. If it is too short, too long, or mistimed, accumulator values leak from the previous run into the next one — a silent wrong-answer bug. This is exactly the class of bug Category 3's weight-poison sequences (in progress) are designed to catch.

## A.7 Timing Diagram (cycle-by-cycle, N=2 example, as-built)

This replaces v1.0's A.6 diagram, which showed the old horizontal-flow 2N timing. This one reflects the actual RTL timing for a 2x2 computation (N=2, active\_dim=2):

| Signal    | 0    | 1     | 2     | 3    | 4     | 5                            |
|-----------|------|-------|-------|------|-------|------------------------------|
| State     | IDLE | WLOAD | WLOAD | PCLR | AFLOW | AFLOW → DONE                 |
| start     | 1    | 0     | 0     | 0    | 0     | 0                            |
| pe_clear  |      |       |       | 1    |       |                              |
| row 0 fed |      |       |       |      | row 1 | row 2                        |
| done      |      |       |       |      |       | 1 (at flow_last = dim+N = 4) |

*WEIGHT\_LOAD runs for N=2 cycles here (cycles 1–2) since WEIGHT\_LOAD\_CYCLES = N regardless of active\_dim. ACTIVATION\_FLOW's internal counter runs from 0 to flow\_last = active\_dim + N = 4, so DONE is reached on the 5th flow cycle. Measured start-to-done span for this example is 6 cycles — matching the dim+5-at-N=4 pattern noted in Part C (here N=2, so it is not directly dim+5, but the same active\_dim+N+1 formula: 2+2+1=5 ACTIVATION\_FLOW-relative cycles, plus the N=2 WEIGHT\_LOAD and 1 PE\_CLEAR cycle before it).*

## A.8 Definition of Done (for the design)

Unchanged in spirit from v1.0, restated against the DiP implementation:

- A computation, once started, drives the FSM IDLE → WEIGHT\_LOAD → PE\_CLEAR → ACTIVATION\_FLOW → DONE without getting stuck.
- pe\_clear asserts for exactly one cycle in PE\_CLEAR and zeros all accumulators.
- For any N from 1 to 4, the output buffer holds the correct int32 matrix-multiply result, matching the Python reference model (permutation and diagonal routing are purely internal — the external contract, A = activations in, B = weights in, C = AxB out, is unchanged by DiP).
- STATUS\_REG done reads 1 only when results are actually valid; the testbench can poll it safely.

- **PENDING:** back-to-back computations produce correct results with no accumulator leakage between runs, and a mid-computation reset returns the FSM to IDLE cleanly — both are the subject of Category 3 and 4, which are in progress, not yet complete.

## A.9 Signal Map and Port Lists

Port names and widths are unchanged from v1.0's A.9 for every module except `systolic_array.sv`, whose ports are restated below to reflect the DiP row-based activation interface (a full row per cycle, not a per-PE column feed).

| Port             | Dir | Width | Meaning (DiP)                                                                                                                                                                                                                           |
|------------------|-----|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| activations      | in  | N×8   | One FULL ROW of matrix A, fed to row 0 in parallel every active cycle (was: one column trickling in, one element per row, in v1.0).                                                                                                     |
| weights          | in  | N×N×8 | Natural (unpermuted) weight matrix. Permutation (row+col mod N) is applied internally.                                                                                                                                                  |
| load_weight      | in  | 1     | Fanned to all PEs, unchanged.                                                                                                                                                                                                           |
| pe_clear         | in  | 1     | Fanned to all PEs, unchanged.                                                                                                                                                                                                           |
| flow_en          | in  | 1     | Gates the row-0 external entry every cycle (was: gated per-row activation feed in v1.0).                                                                                                                                                |
| accum_bottom_row | out | N×32  | Raw, still-accumulating bottom accumulator row. NOT yet a finished result matrix — <code>deskew_capture.sv</code> owns turning this into results (new indirection vs v1.0, where the array's own results port was the finished output). |

*pe.sv, mmu\_controller.sv, and axi\_lite\_slave.sv port lists are unchanged from v1.0 A.9 and are not restated here.*

## A.10 Correction: Accumulator Overflow Bound

### MINOR CORRECTION vs v1.0

v1.0 stated the worst-case MAC product as  $127 \times 127$ . `pe.sv`'s own header comment and formal proof use the tighter true bound: `activation_in`'s magnitude can reach 128 (since `int8` signed range is  $-128..127$ ), so the worst-case single product magnitude is  $128 \times 127 = 16,256$ , not  $127 \times 127 = 16,129$ . Accumulated over  $N=4$  terms this is  $4 \times 16,256 = 65,024$  — still trivially within `int32` range ( $\sim 2.1$  billion), so Proof 1's conclusion (PROVEN) is unaffected. This is a documentation precision fix, not a design change.



## Part B — Register (RAL) Spec

Unchanged from v1.0. DiP is entirely internal to the datapath; the AXI-Lite control surface was never touched by the architecture change. Restated here in full so this document remains self-contained.

### B.1 What 'RAL' means (plain language)

Register Abstraction Layer. Instead of a test driving raw bus wires to poke a register, the RAL model lets a test say something readable like `reg_model.DIM_REG.write(4)` and the framework handles the bus details.

### B.2 Register Map

| Register   | Offset | Direction    | Width | Reset | Purpose                                                      |
|------------|--------|--------------|-------|-------|--------------------------------------------------------------|
| DIM_REG    | 0x0    | Write / Read | 3-bit | 0x0   | Set matrix dimension N (1–4) for this computation.           |
| CTRL_REG   | 0x4    | Write / Read | 1-bit | 0x0   | Bit 0 = start. Triggers weight load then activation flow.    |
| STATUS_REG | 0x8    | Read-only    | 1-bit | 0x0   | Bit 0 = done. Poll until 1 before reading the output buffer. |

### B.3 Locked Register Rules

- STATUS\_REG is strictly read-only. A write to it must not change its value (formally proven — Proof 2, Part D).
- DIM\_REG only accepts N in the range 1–4. Driving 0 or 5–7 is illegal stimulus used by error-injection tests (TC-024, TC-026); the design must not produce a spurious result.
- Writing start = 1 in CTRL\_REG when the FSM is not IDLE is illegal (premature/double start, TC-024/TC-025). The negative scoreboard path confirms no spurious output.
- done in STATUS\_REG reads 1 only when the output buffer genuinely holds valid results.
- All three registers reset to 0.

## Part C — Performance Spec

### OPEN ITEM — THIS ENTIRE SECTION IS THE CENTRAL DISCREPANCY THIS REVISION EXISTS TO SURFACE

v1.0 locked latency at  $2N$  cycles. The RTL as built, the controller's own SVA, and a scoreboard removal note all independently agree the actual measured latency is  $\text{active\_dim} + N + 1$  — and `mmu_perf_checker.sv` (D1) was never updated to match. This is not resolved by this document. It needs Samarth, Jad, and whoever owns C.6 (per the original assignment) to make a decision, per the original Rule 15/16 change-control process. What follows documents the evidence precisely so that decision can be made from facts, not guesswork.

### C.1 The Three Numbers in Play

| Source                                                                                        | Formula stated                                           | N=4 value           | Where found                                                                               |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------------|
| v1.0 spec (locked contract)                                                                   | $2N$                                                     | 8 cycles            | docs/SpecDoc.pdf, C.2/C.6                                                                 |
| <code>mmu_controller.sv</code> +<br><code>mmu_controller_sva.sv</code> (B3) —<br>RTL AS BUILT | $\text{active\_dim} + N + 1$                             | 9 cycles<br>(dim=4) | rtl/mmu_controller.sv<br>header comment;<br>sva/mmu_controller_sva.sv<br>p_result_latency |
| <code>mmu_perf_checker.sv</code> (D1) —<br>STALE, UNCHANGED                                   | $2N$ (hardcoded,<br><code>exp_latency()</code> function) | 8 cycles            | perf/mmu_perf_checker.sv,<br>function <code>exp_latency()</code>                          |

A fourth data point exists and is worth surfacing even though it slightly disagrees with the "as-built" number above: `mmu_formal.sv`'s SPEC NOTE 2 describes DiP's "true" theoretical latency as  $2N-1$ , distinct from what `mmu_controller.sv` currently implements (which that same note describes as sized for the old " $2N-1$  unpipelined + 1 pipeline =  $2N$ " window, i.e.  $\text{flow\_last} = 2 \times \text{dim} - 1$ ). This is a third possible target, separate from the  $\text{dim} + N + 1$  figure the scoreboard's removal note measured directly against simulation. All three formulas ( $2N$ ,  $2N-1$ ,  $\text{dim} + N + 1$ ) currently have at least one repo artifact pointing at them; only  $\text{dim} + N + 1$  is what the RTL currently, measurably does.

### C.2 What the Scoreboard Actually Measured

`mmu_scoreboard.sv`'s `latency_checker` was removed on 2026-07-26 after firing on every single pass — not because it was miscounting, but because the  $2N$  contract it was checking against was wrong. Its removal note, preserved verbatim in the file, is the most direct evidence available:

```
UVM_ERROR [LAT_CHK] dim=4: latency 9 cycles, contract requires exactly 2N (8)
```

```
Measured latency, dim=1..4:
```

```
dim | 1  2  3  4
cyc | 6  7  8  9
```

```
i.e. latency = dim + 5 (equivalently: active_dim + N + 1, at N=4)
```

The note traces the root cause structurally, not as a bug: `mmu_controller.sv` sizes `ACTIVATION_FLOW` as  $\text{flow\_last} = \text{dim} + N$ ; output row  $r$  settles on the bottom accumulator at flow cycle  $N + r$ ; and `deskew_capture.sv`'s `flow_cycle` counter lags its input by one cycle, so the last row is captured on the very last flow cycle — the window is exactly tight, not off-by-one. Forcing the window to  $2N$  would need  $\text{dim} + 4$  flow cycles instead of  $\text{dim} + N$ , which only coincides with the current sizing at  $\text{dim} = 4$  and truncates the capture window for  $\text{dim} = 1..3$ .

*The monitor still measures latency into every transaction's `data_txn.latency` field even though nothing currently asserts on it — the numbers are available for whoever picks this back up.*

### C.3 Two Checkers, Currently Disagreeing

| Check                                          | Owner   | Level              | Current state                                                                                                                                                                                                                       |
|------------------------------------------------|---------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| latency_checker<br>(mmu_scoreboard.sv)         | Samarth | Transaction-level  | REMOVED 2026-07-26 — was correctly measuring $\text{dim}+5$ against an incorrect $2N$ contract. No replacement checker re-added yet pending the contract decision.                                                                  |
| mmu_perf_checker.sv / D1<br>(bound to mmu_top) | Jad     | Signal-level (SVA) | STILL PRESENT, STILL ASSERTS $2N$ . Will fire on every legal computation once run, since actual latency is $\text{dim}+N+1 \neq 2N$ for all $N$ . Header comment inside the file already flags this as an open reconciliation item. |

Until the contract question is resolved, running `mmu_perf_checker.sv` against current RTL should be expected to produce false-failing D1 violations — this is a known, explained state, not a new bug to chase.

### C.4 What `perf_report.md` Must Contain (once C.1's decision lands)

Unchanged in structure from v1.0 — only the expected-value column changes once the contract is chosen:

- A table of actual vs expected cycle counts for every matrix dimension  $N = 1, 2, 3, 4$  — expected column populated from whichever formula wins the C.1 decision.
- The back-to-back throughput result: computations run, measured cycles-per-result vs theoretical best.
- A short explanation of why the latency formula is what it is — for DiP, this should describe the diagonal-propagation term ( $N$  cycles) and the deskew\_capture one-cycle lag, not the old "2 contributions of  $N$  cycles each" framing from v1.0 C.6, which was written for the horizontal-flow design and no longer describes what the RTL does.
- Any deviation between actual and expected, with root cause and fix, cross-referenced to `BUGS.md`.
- A one-line pass/fail statement against whichever contract is locked.

### C.5 Recommended Next Step (not a decision — a process note)

Per the original Rule 16 ("Samarth's scoreboard `latency_checker` and Jad's `mmu_perf_checker.sv` must agree and are kept on purpose as defense-in-depth") and Rule 15's change-control list, resolving this needs one PR that updates, together: this section (C.1–C.4), `mmu_perf_checker.sv`'s `exp_latency()` function, a re-added scoreboard `latency_checker`, `perf_report.md`'s expected column, and the README's latency claim. Until that PR lands, treat any latency-related test result (Category 6, D1) as informative rather than gating.

## Part D — Formal Verification Spec

Owner: Atharva (verification lead) — sole team member with JasperGold training. Tool: Cadence JasperGold, run exclusively through JasperGold, never through Xcelium.

### D.1 Proof 1 — Accumulator Overflow Impossibility

| Field                  | Value                                                                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| File / bound to        | formal/pe_formal.sv → pe.sv                                                                                                                     |
| TCL                    | formal/pe_overflow.tcl                                                                                                                          |
| Result                 | PROVEN                                                                                                                                          |
| Evidence               | formal/results/pe_overflow_report.txt                                                                                                           |
| Bound corrected (A.10) | Worst-case product $128 \times 127 = 16,256$ (not $127 \times 127$ ); 4-term accumulation max 65,024 — unaffected, still far below int32 range. |

Note: this proof is bound to pe.sv, which was structurally unchanged by the DiP rewrite (A.2) — the proof's validity is unaffected by the interconnect change outside the PE.

### D.2 Proof 2 — AXI-Lite Protocol Compliance

| Field           | Value                                    |
|-----------------|------------------------------------------|
| File / bound to | formal/axi_formal.sv → axi_lite_slave.sv |
| TCL             | formal/axi_compliance.tcl                |
| Result          | PROVEN                                   |
| Evidence        | formal/results/axi_compliance_report.txt |

#### **CORRECTION vs v1.0's stated A2 property**

v1.0 (and the earlier UVM plan PDF) stated that A2 required awvalid and wvalid to assert on the SAME cycle. That was never a real AXI-Lite requirement — AXI-Lite's write-address and write-data channels are independent. sva/axi\_lite\_sva.sv's current header explicitly notes this correction: the

| Field                                                                                                                                                                                                                                                                                                                    | Value |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| property now checks wvalid/wdata stability the same way A1 checks awvalid/awaddr stability (once wvalid rises, it and wdata must stay stable until wready), rather than requiring simultaneity. Any test plan language describing A2 as a "simultaneous assertion requirement" is describing the old, incorrect version. |       |

### D.3 Proof 3 — 2x2 Functional Correctness

| Field           | Value                                                                                  |
|-----------------|----------------------------------------------------------------------------------------|
| File / bound to | formal/mmu_formal.sv → mmu_top.sv (DIM_REG = 2)                                        |
| TCL             | formal/twobytwo_correct.tcl                                                            |
| Result          | PROVEN (bounded — see results/PROOF3_TIMELINE.md and twobytwo_correct_FINALREPORT.txt) |
| Evidence        | formal/results/CompletedProof3_Screenshot.jpg, twobytwo_correct_FINALREPORT.txt        |

Two things worth carrying forward from mmu\_formal.sv's own header comments, since they bear directly on open items elsewhere in this document:

- **State-space correction:** v1.0's prose describing the input space needs correcting to  $2^{64}$  (four int8 activations + four int8 weights = 64 free bits), not  $2^{32}$  as originally written.
- **Deliberately latency-agnostic:** this proof checks the VALUE of results on whichever cycle done asserts — it never asserts a specific cycle count. This means Proof 3's PROVEN status holds regardless of how the Part C latency OPEN ITEM is eventually resolved; it is not evidence for either side of that question, and should not be cited as such.

### D.4 C1 — zero\_input\_no\_accumulate: Not Yet Started

#### OPEN ITEM

pe\_sva.sv, the file that binds C1 (zero\_input\_no\_accumulate) to every PE instance, does not exist in the repository yet. This affects: the Category 1 positive-mode exercise mapping for TC-005/TC-010/TC-014 (those

tests still pass functionally via the scoreboard, but nothing SVA-level is watching the zero-activation accumulator invariant), and TC-035f (the force-injection test designed to prove C1 actually fires) — there is currently no assertion for it to fire.

## D.5 What Distinguishes Formal From the SVA Simulation Assertions

Unchanged from v1.0: the SVA assertions in `sva/` check properties on the specific inputs the UVM sequences generate. The JasperGold formal proofs make no assumptions about which inputs arrive — the tool exhaustively explores all reachable states and either proves a property holds universally or produces a counterexample. This is why formal is the right tool for overflow guarantees, protocol invariants, and functional correctness.

## Appendix — Revision Change Log

| #  | Change                                                                                                                                                                         | Section      |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 1  | Architecture replaced: horizontal weight-stationary flow → DiP diagonal-input, permuted weight-stationary                                                                      | A.2          |
| 2  | skew_buffer.sv removed from instantiation; deskew_capture.sv added as its replacement (output-side capture, not input-side stagger)                                            | A.4          |
| 3  | systolic_array.sv port semantics restated for row-based (not per-element) activation feed                                                                                      | A.9          |
| 4  | Overflow bound corrected: $128 \times 127 = 16,256$ worst-case product (was stated as $127 \times 127 = 16,129$ )                                                              | A.10         |
| 5  | Latency contract changed from LOCKED (2N) to OPEN ITEM (active_dim + N + 1 as measured; 2N-1 also referenced by one formal comment; 2N still hardcoded in mmu_perf_checker.sv) | Part C (all) |
| 6  | Scoreboard latency_checker removed 2026-07-26 pending contract resolution                                                                                                      | C.2, C.3     |
| 7  | A2 (axi_wvalid_with_awvalid) property corrected: no longer requires simultaneous awvalid/wvalid                                                                                | D.2          |
| 8  | Proof 3 state-space language corrected: $2^{64}$ , not $2^{32}$                                                                                                                | D.3          |
| 9  | C1 (zero_input_no_accumulate / pe_sva.sv) status downgraded from v1.0's implicit "complete" framing to NOT STARTED                                                             | D.4          |
| 10 | UVM Category 3 and 4 test-class files flagged IN PROGRESS, not complete, per team status                                                                                       | 0.3          |
| 11 | D2 (back_to_back_throughput SVA) flagged NOT WRITTEN; TC-035g has no current target                                                                                            | 0.3          |

## Glossary

| Term                    | Plain meaning                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Systolic array          | A grid of tiny multiply-add units that pass data to neighbors each cycle instead of using memory.                                                                                                                          |
| PE (processing element) | One cell in the grid. Does one multiply-add per cycle.                                                                                                                                                                     |
| Weight-stationary       | Weights are loaded once and held still; activations flow past them.                                                                                                                                                        |
| DiP                     | Diagonal-input, Permuted weight-stationary — the current dataflow. Row 0 is fed externally every cycle; data moves diagonally down-and-across; weights are permuted by $(\text{row} + \text{col}) \bmod N$ before loading. |
| MAC                     | Multiply-Accumulate: $\text{total} = \text{total} + (a \times b)$ . The one operation a PE does.                                                                                                                           |
| int8 / int32            | 8-bit / 32-bit signed integers. Inputs are int8; the running total is int32 to avoid overflow.                                                                                                                             |
| AXI-Lite                | A simple industry-standard control bus used to read/write the three registers.                                                                                                                                             |
| RAL                     | Register Abstraction Layer — lets tests use register names instead of raw bus wires.                                                                                                                                       |
| FSM                     | Finite State Machine — the controller that steps through IDLE → ... → DONE.                                                                                                                                                |

| Term           | Plain meaning                                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------------------------------|
| pe_clear       | A signal that zeros every PE accumulator for exactly one cycle before a new run.                                             |
| Latency        | Cycles from activations starting to flow until the result is ready. OPEN ITEM — see Part C.                                  |
| deskew_capture | New module that snapshots the array's raw bottom accumulator row into a finished result matrix at the correct per-row cycle. |
| SVA            | SystemVerilog Assertions — rules checked automatically every cycle during simulation.                                        |
| DUT            | Design Under Test — the thing being verified. Here, mmu_top.sv.                                                              |